

Turtle ID

Kaplumbaga Fotograf Tanim Sistemi

Yusuf Altunbay - 22253072

-
- 1. Projenin Amaci*
 - 2. Coklu Ajan Mimarisi*
 - 3. SOLID Prensipleri*
 - 4. Clean Code*
 - 5. Testler*

1. Projenin Amaci

Bu proje, kaplumbagalari fotograftan taniyan bir masaustu uygulamasidir. Farkli zamanlarda, farkli isik kosullarinda cekilen iki fotografin ayni kaplumbagaya ait olup olmadigini benzerlik skoru uzerinden tespit eder.

EfficientNet-B0 modeli her fotograftan 1280 sayilik bir parmak izi (embedding vektörü) cikarir. Iki vektorun cosine benzerligi 0.82 esigini gecerse esleme basarili sayilir.

Kullanilan Teknolojiler

Teknoloji	Kullanım Amaci
Python 3.11	Ana programlama dili
PySide6	Masaustu kullanıcı arayuzu
EfficientNet-B0	Fotograftan özellik vektörü çıkarma
OpenCV	Görüntü ön işleme
SQLite / SQLAlchemy	Veri saklama
pytest	Otomatik testler (41 test)

2. Çoklu Ajan Mimarisi

Sistem 6 ajandan oluşur. Her ajan tek bir işi yapar ve EventBus aracılığıyla diğer ajanlarla haberleşir. Ajanlar birbirini doğrudan çağırılmaz, bu da kodu bağımsız ve test edilebilir kılar.

Ajan	Görevi
ConfigAgent	Ayarları (esik değeri, top-k) okur ve yazar
PhotoValidationAgent	Fotografin net ve geçerli olduğunu kontrol eder
ImageAgent	Fotografın embedding vektörü üretir
MatchingAgent	Sorgu vektörünü veritabanıyla karşılaştırır
DataAgent	Kaplumbaga kayıt ve listeleme işlemleri
UIAgent	Ekranları yönetir, butonları iş akışlarına bağlar

EventBus Örneği

Ajanlar birbirine mesaj gönderir. Aşağıda eşleme tamamlandığında doğrulama logunun otomatik olarak kaydedilmesi gösterilmiştir:

```
# MatchingAgent eşlemeyi tamamlayınca mesaj yayımlar:
self.event_bus.publish(ESLEME_BULUNDU, sonuc)

# DataAgent bu mesajı dinler, kendisi otomatik log kaydeder:
class DataAgent:
    def baslarken(self):
        self.event_bus.subscribe(ESLEME_BULUNDU, self.log_kaydet)

    def log_kaydet(self, sonuc):
        veritabani.dogrulama_logu_kaydet(sonuc)
```

3. SOLID Prensipleri

S — Tek Sorumluluk (Single Responsibility)

Her sınıf tek bir işi yapar. PhotoValidationAgent sadece fotoğraf kontrolü yapar; veritabanına yazmaz, esleme yapmaz.

```
class PhotoValidationAgent:
    def validate(self, fotograf_yolu):
        self._dosya_var_mi(yol)
        self._uzanti_uygun_mu(yol)
        self._bulanik_mi(goruntu)
        self._karanlık_mi(goruntu)
        return ValidationResult(...)
```

O — Açık/Kapalı (Open/Closed)

Yeni özellik eklemek için mevcut kodu değiştirmek gerekmez. Örneğin farklı bir esleme algoritması kullanmak istersek sadece container.py'da tek satır değişir.

```
# Mevcut: eslestirici = CosineMatcher()
# Yeni algoritma için:
eslestirici = FAISSMatcher() # Baska hicbir dosya degismez
```

D — Bağımlılık Tersine Çevrilmesi (Dependency Inversion)

Ajanlar somut sınıflar yerine soyut arayuzlere bağımlidir. Bu sayede testlerde gerçek model yerine sahte bir nesne kullanılabilir.

```
# Kötü: ajan somut sinifi olusturuyor
class ImageAgent:
    def __init__(self):
        self.model = TimmEfficientNetExtractor()

# İyi: arayuz disaridan enjekte ediliyor
class ImageAgent:
    def __init__(self, model: IEmbeddingExtractor):
        self.model = model
```

4. Clean Code

Anlamli Isimler

Her sinif ve metod ismi ne yaptigini dogrudan anlatir.

```
PhotoValidationAgent  # Fotograflari dogrular
CosineMatcher         # Cosine benzerligi ile eslestirir
find_best_match()     # En iyi eslemeyi bulur
has_embeddings()       # "Embedding var mi?" sorusunu sorar
```

Kucuk Fonksiyonlar

Uzun bir fonksiyon yerine, her biri tek isi yapan kucuk fonksiyonlar tercih edilir.

```
# Kotu: her seyi yapan tek buyuk fonksiyon
def isle(yol):
    # 80 satir ...

# Iyi: her kontrolun kendi fonksiyonu var
def validate(self, yol):
    self._dosya_var_mi(yol, sonuc)
    self._uzanti_kontrol(yol, sonuc)
    self._bulaniklik_kontrol(goruntu, sonuc)
    return sonuc
```

Hata Yonetimi

Hatalar exception firlatmak yerine sonuc nesnesiyle kullaniciya iletilir. EventBus'ta bir abonenin hatasi diger aboneleri durdurmaz.

```
if not dogrulama.gecerli:
    return KayitSonucu(basarili=False, hatalar=dogrulama.hatalar)

# EventBus'ta hata izolasyonu:
for abone in aboneler:
    try:
        abone(olay, veri)
    except Exception as hata:
        logger.error(hata) # Diger aboneler calismaya devam eder
```

5. Testler

Proje 41 otomatik testle dogrulanmistir, tamami gecmektedir.

Test Dosyasi	Kapsam	Adet
test_event_bus.py	Pub/sub, thread guvenligi	7
test_matcher.py	Cosine esleme, esik mantigi	7
test_models.py	Veri modelleri	13
test_validation_agent.py	Fotograf dogrulama	4
test_repository.py	Veritabani CRUD	10
Toplam		41

Ornek test:

```
def test_ayni_vektor_eslesmeli(matcher):
    v = normalize(random_vector(128))
    sonuc = matcher.find_best_match(v, [Embedding(vector=v)], threshold=0.9)
    assert sonuc.is_match == True
    assert sonuc.score == pytest.approx(1.0)

def test_bos_liste_eslememeli(matcher):
    sonuc = matcher.find_best_match(v, [], threshold=0.75)
    assert sonuc.is_match == False
```

Pipeline Testi Sonuclari

Sentetik fotoğraflar üzerinde yapılan testte:

- Ayni kaplumbaga (karanlik fotograf): benzerlik 0.875 — esleme basarili
- Ayni kaplumbaga (parlak fotograf): benzerlik 0.962 — esleme basarili
- Farkli kaplumbaga: benzerlik 0.414 — esleme yok

Yanlis pozitif esleme: 0 / 10 karsilastirma